Agent 行为追踪 ID 体系设计文档

一、概述

1.1 背景

在 Agent 系统中，为了实现行为可追溯、风险可控、数据血缘可追踪，需要建立一套完整的 ID 体系。该体系需要解决两个核心问题：

1. **执行链追踪**：谁在什么时候做了什么
2. **数据血缘追踪**：这条数据从哪来、经过了什么、流向哪

1.2 设计目标

- **全链路可追溯**：从 Agent 实体到单次工具调用，实现完整的行为追踪
- **数据血缘不断裂**：数据在 Agent 间流转时，血缘关系保持完整
- **污点传播可追踪**：支持 Taint Analysis，识别风险数据流向
- **跨 Agent 关联**：支持 Agent 间任务委派与数据流转的追踪
- **风险量化评估**：基于 ID 聚合实现累积风险评分

二、核心设计思路：双链交织

2.1 双链模型

本体系采用"执行链"与"数据链"双链交织的设计模式：

链类型	追踪维度	核心问题
执行链（Execution Trace）	行为维度	谁在什么时候做了什么
数据链（Data Lineage / Taint）	数据维度	这条数据从哪来、经过了什么、流向哪

2.2 交叉引用机制

每个 Action 同时携带两种 ID：

- **执行 ID**：标识本次操作在执行链中的位置
- **数据血缘 ID**：标识本次操作涉及的数据血缘关系

执行链（Execution Trace） 数据链（Data Lineage / Taint） "谁在什么时候做了什么" "这条数据从哪来、经过了什么、流向哪" | | | 交叉引用 |
└──────────────────────┬──────────────────────────────────────┘ | 每个 Action 同时携带 执行ID + 数据血缘ID

三、分层 ID 结构

3.1 ID 层级定义

本体系采用六层 ID 结构，从组织到数据单元逐层细化：

层级	ID 名称	说明	生命周期
Level 0	`org_id`	组织租户	持久
Level 1	`agent_id`	Agent 实体	持久（跨会话）
Level 2	`session_id`	一次会话/任务	会话期间
Level 3	`turn_id`	一轮 LLM 推理	单次推理
Level 4	`action_id`	一次工具调用/外部动作	单次调用
Level 5	`data_id`	一条数据单元的血缘标识	数据生命周期

3.2 各层 ID 职责

ID 层级	追踪什么
`agent_id`	这个 Agent 是谁（跨会话持久，关联姿态/权限/Owner）
`session_id`	一次完整任务的生命周期边界
`turn_id`	一次 LLM 推理（输入 prompt → 输出 response）
`action_id`	一次具体的工具调用/API 请求/代码执行
`data_id`	一条数据从产生到消亡的全生命周期

四、ID 生成规则

4.1 格式规范

格式：`{type_prefix}_{timestamp}_{random}`

4.2 生成示例

agent_id = "agt_20260327_a3f8c1" # 注册时生成，持久不变 session_id = "ses_20260327T102300_7k2m" # 会话启动时生成 turn_id = "trn_20260327T102301_p9x4" # 每次 LLM 调用时生成 action_id = "act_20260327T102301_q1w2" # 每次工具调用时生成 data_id = "dat_20260327T102301_m5n6" # 数据首次进入系统时生成

4.3 关键原则

> **核心原则**：`data_id` 一旦生成就跟着数据走，不管数据被几个 Agent、几个 Session 处理过。

五、完整事件记录结构

5.1 Action Record 结构

每次 tool 调用产生一条 Action Record，包含以下字段组：

{ "agent_id": "agt_20260327_a3f8c1", "session_id": "ses_20260327T102300_7k2m", "turn_id": "trn_20260327T102301_p9x4", "action_id": "act_20260327T102301_q1w2",

"parent_action_id": null, "parent_agent_id": null,

"action_type": "tool_call", "tool_name": "read_crm_contact", "input": { "contact_id": "C-1234" }, "output": { "name": "...", "email": "...", "notes": "..." }, "timestamp": "2026-03-27T10:23:01.342Z", "duration_ms": 127,

"data_inputs": ["dat_...*m5n6"], "data_outputs": ["dat...*_r7s8"],

"taint": { "inherited_from": ["dat_..._m5n6"], "taint_level": "untrusted", "taint_source": "external_crm_field" },

"risk": { "cumulative_score": 0.35, "delta": 0.12, "flags": ["pii_in_context", "external_data_mixed"] } }

5.2 字段组说明

字段组	包含字段	说明
执行链 ID	`agent_id`, `session_id`, `turn_id`, `action_id`	标识操作在执行链中的位置
父子关系	`parent_action_id`, `parent_agent_id`	支持嵌套 Agent 场景
执行内容	`action_type`, `tool_name`, `input`, `output`, `timestamp`, `duration_ms`	记录具体操作内容
数据血缘链	`data_inputs`, `data_outputs`	记录数据的消费与产出关系
污点标记	`taint.inherited_from`, `taint.taint_level`, `taint.taint_source`	记录污点传播信息
风险快照	`risk.cumulative_score`, `risk.delta`, `risk.flags`	记录累积风险评分

5.3 action_type 枚举值

值	说明
`tool_call`	工具调用
`llm_inference`	LLM 推理
`api_request`	API 请求
`code_exec`	代码执行

六、四种关键追踪能力

6.1 执行链追踪：还原"Agent 做了什么"

查询方式：按 `session_id` 聚合，按 `turn_id` → `action_id` 排序

追踪示例：

session_id = "ses_...7k2m" | ├── turn_id: trn_001 | ├── action_id: act_001 [llm_inference] "理解用户指令" | ├── action_id: act_002 [tool_call] "read_crm_contact" | └── action_id: act_003 [tool_call] "query_order_history" | ├── turn_id: trn_002 | ├──

action_id: act_004 [llm_inference] "综合分析" │ └─── action_id: act_005 [tool_call] "send_email" ← Sink │ └─── turn_id: trn_003 └─── action_id: act_006 [llm_inference] "生成总结报告"

6.2 数据血缘追踪：还原"数据从哪来、到哪去"

核心机制：每条数据携带 `data_id`，每次被 Agent 处理时记录 `data_inputs` → `data_outputs` 的转换关系。

血缘传播示例：

dat_001 (用户原始输入 - untrusted) │ ├──→ act_002 消费 ──→ dat_002 (CRM 数据 + 用户输入 = mixed) │ ├──→ act_003 消费 ──→ dat_003 (订单历史 - trusted) │ └──→ act_004 消费 dat_002 + dat_003 ──→ dat_004 (综合分析 = mixed) │ └──→ act_005 消费 dat_004 ──→ send_email(dat_004) ⚠️ mixed 数据到达 Sink！

查询方式：给定任意 `data_id`，沿 `data_inputs` / `data_outputs` 链正向/反向遍历，即可还原完整数据血缘图。

6.3 污点传播追踪：实现 Taint Analysis

规则引擎实现：

def propagate_taint(action_record): input_taints = [get_taint(d) for d in action_record["data_inputs"]] # 传播规则：只要有一个输入是 untrusted，输出就被标记 if any(t.level == "untrusted" for t in input_taints): for output_id in action_record["data_outputs"]: mark_taint(output_id, level="mixed", inherited_from=action_record["data_inputs"], source=merge_sources(input_taints)) # Sink 检测：被污染的数据到达敏感操作 if action_record["action_type"] in SINK_TYPES: if any(t.level in ("untrusted", "mixed") for t in input_taints): raise_risk_alert(type="tainted_data_at_sink", action=action_record, taint_chain=trace_back(action_record["data_inputs"]))

污点等级定义：

等级	说明
`untrusted`	不可信数据（用户输入、外部数据）
`mixed`	混合数据（trusted + untrusted）
`trusted`	可信数据（系统内部生成）

6.4 跨 Agent 关联：追踪 Agent 间的任务委派与数据流转

场景：Agent A 调用 Agent B 完成子任务

Agent A (agt_001) session: ses_A turn: trn_A_003 action: act_A_007 [delegate_to_agent] │ │ 传递: data_id = dat_004 (mixed) │ parent_action_id = act_A_007 │ ▼ Agent B (agt_002) session: ses_B turn: trn_B_001 action: act_B_001 [tool_call] data_inputs: [dat_004] ← 污点跟着数据跨 Agent 传播 taint.inherited_from: [dat_004] ← 血缘不断

关键机制：

- `parent_action_id` + `parent_agent_id` 实现跨 Agent 的执行链拼接
- `data_id` 实现跨 Agent 的数据血缘不断裂

七、累积风险评分的 ID 聚合

7.1 风险评分算法

def compute_cumulative_risk(session_id): actions = get_actions_by_session(session_id)

```
score = 0.0
for action in actions:
    # 维度 1：本步骤的污点严重性
    taint_delta = assess_taint_severity(action)
    # 维度 2：权限升级（工具能力相对于Agent姿态的偏移）
    priv_delta = assess_privilege_escalation(action, get_agent_posture(action.age

    # 维度 3：数据敏感度（本步骤接触的数据分类等级）
    data_delta = assess_data_sensitivity(action.data_inputs)
    # 维度 4：跨系统跳跃（从一个安全域跳到另一个）
    boundary_delta = assess_boundary_crossing(action, previous_action)
    # 维度 5：行为漂移（与该 Agent 的历史基线相比）
    drift_delta = assess_behavioral_drift(action, get_baseline(action.agent_id))
    delta = weighted_sum(taint_delta, priv_delta, data_delta, boundary_delta, dri
    score += delta
    # 写入 action record
    action.risk = { "cumulative_score": score, "delta": delta }
    # 触发 Beam 干预阈值检查
    if score > THRESHOLD:
        beam_intervene(session_id, action, score)
return score
```

7.2 风险评分维度

维度	说明	评估方法
污点严重性	本步骤涉及的数据污点程度	分析 `data_inputs` 的 `taint_level`
权限升级	工具能力相对于 Agent 姿态的偏移	对比工具权限与 Agent 姿态
数据敏感度	本步骤接触的数据分类等级	分析数据分类标签
跨系统跳跃	从一个安全域跳到另一个	检测系统边界跨越
行为漂移	与该 Agent 的历史基线相比	对比历史行为模式

7.3 Beam 干预机制

当累积风险评分超过阈值时，触发 Beam 干预：

- 注入安全上下文提示
- 限制敏感操作
- 通知管理员审核

八、整体 ID 拓扑视图

org_001 ├── agt_001 (Sales Agent) │ ├── [姿态快照] tools: [crm, email, calendar], data_access: [customer_pii] │ │ │ ├── ses_001 (处理客户咨询) │ │ ├── trn_001 │ │ │ ├── act_001 [llm] ── dat_001(user_input, untrusted) │ │ │ ├── act_002 [crm.read] ── dat_001 → dat_002(crm+input, mixed) │ │ │ └── act_003 [db.query] ── dat_003(order_history, trusted) │ │ │ │ │ ├── trn_002 │ │ │ ├── act_004 [llm] ── dat_002+dat_003 → dat_004(analysis, mixed) │ │ │ └── act_005 [email.send] ── dat_004 → ⚠️ SINK (mixed→external) │ │ │ risk: { cumulative: 0.72, flags: [pii_mixed_external] } │ │ │ → Beam 触发：注入 "不要在邮件中包含客户联系方式" 上下文 │ │ │ │ │ └── trn_003 │ │ └── act_006 [email.send] ── dat_005(sanitized, trusted) ✓ │ │ risk: { cumulative: 0.72 → 0.45 (Beam干预后下降) }

九、总结

本 ID 体系通过"双链交织"的设计，实现了 Agent 行为的全链路追踪能力：

1. **执行链**：通过 `agent_id` → `session_id` → `turn_id` → `action_id` 的层级结构，完整记录 Agent 的行为轨迹
2. **数据链**：通过 `data_id` 及其 `data_inputs` / `data_outputs` 关系，实现数据血缘的完整追踪

3. **污点传播**：通过 `taint` 字段记录污点传播路径，支持 Taint Analysis
4. **跨 Agent 关联**：通过 `parent_action_id` 和 `parent_agent_id` 实现 Agent 间任务委派的追踪
5. **风险量化**：通过多维度风险评分和累积机制，实现风险的动态评估与干预

该体系为 Agent 系统的安全审计、行为分析、风险控制提供了坚实的技术基础。